

IOS_REFERENCE

Core Definition

<ios> defines the foundation of the stream hierarchy: `ios_base` (format flags, precision, width, locale, callbacks, extensible storage) and `basic_ios` (stream state, buffer association, fill character, tied stream). It also declares the unparameterized manipulators (`hex`, `fixed`, `boolalpha`, ...) and the `io_errc` error category.

Every stream — file, string, console — inherits everything documented here. This is the layer that answers "what does `failbit` actually mean" and "where does `precision` live".

#CPP #IOS #IOS_BASE #STREAM-STATE #FMTFLAGS #IOSTATE #OPENMODE #LOCALE

COMPLETE IOS QUICK REFERENCE

```
// =====
// FORMAT FLAGS – type ios_base::fmtflags
// =====
std::ios_base::boolalpha      // bool as true/false
std::ios_base::showbase       // 0x / 0 prefix for hex / octal
std::ios_base::showpoint      // Always emit the decimal point
std::ios_base::showpos        // + on non-negative numbers
std::ios_base::skipws         // Skip leading whitespace on input (DEFAULT ON)
std::ios_base::unitbuf        // Flush after every output operation
std::ios_base::uppercase      // Uppercase hex digits and exponent markers

std::ios_base::dec            // Decimal (DEFAULT)
std::ios_base::oct            // Octal
std::ios_base::hex            // Hexadecimal
std::ios_base::basefield      // = dec | oct | hex (mask)

std::ios_base::fixed          // Fixed-point notation
std::ios_base::scientific     // Scientific notation
std::ios_base::floatfield     // = fixed | scientific (mask)
// both set → hexfloat; neither → default/general

std::ios_base::left           // Pad on the right
std::ios_base::right          // Pad on the left (DEFAULT)
std::ios_base::internal       // Pad after the sign / base prefix
std::ios_base::adjustfield    // = left | right | internal (mask)

// =====
// STREAM STATE – type ios_base::iostate
// =====
std::ios_base::goodbit        // 0 – no errors
std::ios_base::eofbit         // End of input reached
std::ios_base::failbit        // Logical / formatting error (recoverable)
std::ios_base::badbit         // Stream integrity lost (unrecoverable)

// =====
// OPEN MODE – type ios_base::openmode
// =====
std::ios_base::in             // Read
std::ios_base::out            // Write
std::ios_base::app            // Append – all writes forced to the end
std::ios_base::ate            // Seek to the end once on open
std::ios_base::trunc          // Discard existing contents
std::ios_base::binary         // No newline / EOF translation
std::ios_base::noreplace      // C++23 – fail if the file exists

// =====
```

```

// SEEK DIRECTION – type ios_base::seekdir
// =====
std::ios_base::beg           // From the beginning
std::ios_base::cur           // From the current position
std::ios_base::end           // From the end

// =====
// ios_base MEMBER FUNCTIONS – Target | Operation | Output
// =====
s.flags()                   // none      | Get all flags      | Returns fmtflags
s.flags(fl)                  // fmtflags | Replace all flags  | Returns the PREVIOUS flags
s.setf(fl)                   // fmtflags | OR flags in       | Returns previous flags
s.setf(fl, mask)             // fl + mask | Clear mask, set fl | Returns previous flags
s.unsetf(fl)                 // fmtflags | Clear flags       | Returns void
s.precision()                // none     | Get precision     | Returns streamsize (default 6)
s.precision(n)               // count    | Set precision     | Returns previous streamsize
s.width()                     // none     | Get field width   | Returns streamsize
s.width(n)                   // count    | Set field width   | Returns previous; RESET after next
    output
s.imbue(loc)                  // locale   | Set the locale    | Returns previous locale
s.getloc()                    // none     | Get the locale    | Returns locale

// =====
// ios_base – EXTENSIBLE STORAGE & CALLBACKS (for custom stream state)
// =====
std::ios_base::xalloc()       // static   | Reserve an index  | Returns unique int index
s.iword(idx)                  // index    | long& slot        | Per-stream user integer
s.pword(idx)                  // index    | void*& slot        | Per-stream user pointer
s.register_callback(fn, idx)   // fn + idx | Register event hook | Called on copyfmt/imbue/erase
// Event types: ios_base::erase_event, imbue_event, copyfmt_event

// =====
// ios_base – SYNCHRONIZATION & NESTED TYPES
// =====
std::ios_base::sync_with_stdio([b]) // Query/set C stdio sync | Returns previous bool
std::ios_base::failure              // Exception type thrown on error (derives from system_error)
std::ios_base::Init                  // Static init helper for cin/cout (implicit via <iostream>)

// =====
// basic_ios MEMBER FUNCTIONS
// =====
s.good() / s.eof() / s.fail() / s.bad() // State queries | Returns bool
s.rdbuf()                               // none          | All state flags | Returns iostate
s.clear([state])                         // iostate       | Set state exactly | Returns void (default goodbit)
s.setstate(state)                       // iostate       | OR flags into state | Returns void
s.exceptions()                          // none          | Get exception mask | Returns iostate
s.exceptions(mask)                      // iostate       | Throw on these flags | Returns void
explicit operator bool()                 // none          | !fail()          | Enables 'if (s)' / 'while (s >> x)'
s.operator!()                           // none          | fail()           | Returns bool
s.rdbuf()                               // none          | Get stream buffer | Returns basic_streambuf*
s.rdbuf(sb)                             // buf*          | Set stream buffer | Returns previous buffer*
s.tie()                                 // none          | Get tied ostream | Returns basic_ostream*
s.tie(os)                               // ostream*      | Set tied ostream | Returns previous
s.fill()                                // none          | Get fill character | Returns char_type
s.fill(ch)                              // char          | Set fill character | Returns previous char_type
s.copyfmt(other)                        // stream        | Copy ALL formatting | Returns *this; not buffer or state
s.narrow(c, dflt)                       // char_type     | Widen → narrow char | Returns char
s.widen(c)                              // char          | Narrow → char_type | Returns char_type

// =====
// ERROR CATEGORY (C++11)
// =====
std::io_errc::stream           // The single io_errc enumerator
std::istream_category()        // Returns the error_category for stream errors
std::make_error_code(io_errc) // → std::error_code
std::make_error_condition(io_errc) // → std::error_condition

// =====
// MANIPULATORS DECLARED HERE (see IOMANIP_REFERENCE for full coverage)
// =====
boolalpha / noboolalpha          showbase / noshowbase
showpoint / noshowpoint          showpos / noshowpos

```

```

skipws / noskipws           uppercase / nouppercase
unitbuf / nunitbuf         internal / left / right
dec / oct / hex             fixed / scientific / hexfloat / defaultfloat

// =====
// TYPE ALIASES
// =====
std::streamoff              // Signed offset type
std::streamsize             // Signed size/count type
std::streampos              // = fpos<mbstate_t> - absolute position
std::wstreampos / std::ul64streampos / std::u32streampos
std::fpos<State>            // Position type template
std::ios / std::wios        // = basic_ios<char> / basic_ios<wchar_t>

```

COMMON PATTERNS & EXAMPLES

Correct Flag Manipulation Within a Group

```

#include <iostream>

// WRONG - leaves dec set alongside hex; behavior is undefined
os.setf(std::ios::hex);

// RIGHT - clear the whole basefield first
os.setf(std::ios::hex, std::ios::basefield);
os.setf(std::ios::left, std::ios::adjustfield);
os.setf(std::ios::fixed, std::ios::floatfield);

// Or just use the manipulators, which do this for you:
os << std::hex << std::left << std::fixed;

```

Saving and Restoring All Formatting

```

// One-liner reset to defaults:
os.copyfmt(std::ios(nullptr));

// Full save/restore via copyfmt:
std::ios saved(nullptr);
saved.copyfmt(os);           // Snapshot
os << std::hex << std::setprecision(12) << value;
os.copyfmt(saved);           // Restore

```

`copyfmt` copies flags, precision, width, fill, locale, tied stream, exception mask, and all `word` slots. It does **not** copy the stream buffer or the error state.

Interpreting Stream State

```

void report(const std::ios& s) {
    auto st = s.rdstate();
    if (st == std::ios::goodbit) { /* fine */ }
    if (st & std::ios::eofbit) { /* input exhausted */ }
    if (st & std::ios::failbit) { /* format error - recoverable via clear() */ }
    if (st & std::ios::badbit) { /* device error - do not continue */ }
}

// Note: a failed extraction at end of input sets BOTH eofbit and failbit.

```

Exception Mode

```
std::ifstream in;
in.exceptions(std::ios::badbit);           // Throw only on real errors
// in.exceptions(std::ios::failbit | std::ios::badbit); // Also on format errors
// Do NOT include eofbit – normal end of input would throw

try {
    in.open("data.txt");
} catch (const std::ios_base::failure& e) {
    // C++11+: failure derives from system_error
    std::cerr << e.what() << " code=" << e.code() << '\n';
}
```

Per-Stream Custom State with `xalloc` / `iword`

```
#include <iostream>

// Give every stream an "indent level" of our own
inline int indentIndex() {
    static const int idx = std::ios_base::xalloc(); // Allocated once, process-wide
    return idx;
}

std::ostream& indentMore(std::ostream& os) { os.iword(indentIndex()) += 2; return os; }
std::ostream& indentLess(std::ostream& os) { os.iword(indentIndex()) -= 2; return os; }
std::ostream& applyIndent(std::ostream& os) {
    return os << std::string(static_cast<std::size_t>(os.iword(indentIndex())) * ' ');
}

// std::cout << indentMore << applyIndent << "nested\n" << indentLess;
```

`iword` slots default to 0 and are per-stream, so a library can attach state to a caller's stream without touching its interface. `pword` holds a `void*` — register a callback to free it on `erase_event`.

A Callback for Cleanup

```
void onEvent(std::ios_base::event ev, std::ios_base& s, int idx) {
    if (ev == std::ios_base::erase_event) {
        delete static_cast<MyState*>(s.pword(idx)); // Stream being destroyed
    } else if (ev == std::ios_base::copyfmt_event) {
        // pword was shallow-copied – deep copy or null it here
    }
}

// s.register_callback(onEvent, myIndex);
```

`widen` / `narrow` for Character-Type-Generic Code

```
template <typename CharT>
void printline(std::basic_ostream<CharT>& os, const char* ascii) {
    while (*ascii) os.put(os.widen(*ascii++));
    os.put(os.widen('\n'));
}
```

`unitbuf` for Crash-Safe Logging

```
std::ofstream log("app.log");
log << std::unitbuf; // Flush after every insertion – like cerr
// Costs throughput; use only where losing the tail of the log matters.
```

Untying for Speed

```
std::ios_base::sync_with_stdio(false); // Decouple from C stdio buffers
std::cin.tie(nullptr);                // Don't flush cout before each cin read
// After this, do not mix printf/scanf with cin/cout, and expect
// prompts to appear only when cout is actually flushed.
```

IMPORTANT CONCEPTS

`ios_base` VS `basic_ios`

`ios_base` is not a template: it holds everything independent of the character type — format flags, precision, width, locale, `isword/pword`, callbacks. `basic_ios<CharT>` adds what depends on the character type — the stream buffer pointer, the fill character, the tied stream, and the state flags with their conversion operators.

`failbit` VS `badbit`

`failbit` means the operation could not be performed as requested — a parse failure, an open on a missing file, a `getline` that overflowed the buffer. The stream is intact; `clear()` and `retry`. `badbit` means the underlying buffer failed and the stream is no longer trustworthy — a disk error, a broken pipe. Recovery is not expected.

Failed Extraction Sets Both `eof` and `fail`

Reading a number at end of input sets `eofbit` (no more data) and `failbit` (nothing extracted). Testing `eof()` alone is therefore ambiguous; test the operation instead: `while (is >> x)`.

`width()` Resets, Others Persist

Of the three main format quantities, only `width` is consumed by the next formatted output operation. `precision` and `fill` persist until changed. This asymmetry is why `setw` must be repeated for every column.

Flags Within a Mask Are Mutually Exclusive

Setting `hex` without clearing `dec` leaves two `basefield` bits set, which is undefined. Always use the two-argument `setf(flag, mask)` form, or the manipulators, which handle it.

C++11 Changed the Failed-Extraction Value

Before C++11, a failed `>>` left the target untouched. Since C++11 it writes 0 (or the type's zero equivalent). Don't rely on either — check the stream and only use the value on success.

`ios_base::failure` Derives from `system_error`

Since C++11, catching `std::ios_base::failure` gives you a `.code()` in the `iostream_category`. On some standard library versions there was a long-standing ABI split between the C++98 and C++11 versions of this type, which could cause catch blocks to miss — a historical footgun worth knowing about if you see it in older builds.

`sync_with_stdio(true)` Is the Default and It Costs

By default, C++ streams share buffers with C `stdio` so that `printf` and `cout` interleave correctly. Turning that off lets the C++ streams use their own buffering, typically a large speedup for bulk I/O — at the cost of no longer being able to mix the two APIs.

Locale Is Per-Stream

`imbue` affects only that stream. `std::locale::global()` changes the default for streams constructed afterward and for `std::locale()`. Numeric parsing, decimal separators, digit grouping, and `put_money`/`put_time` all route through the imbued locale's facets.

STATE FLAG DECISION TABLE

Situation	good	eof	fail	bad
Fresh stream, nothing read	1	0	0	0
Successful read, more data remains	1	0	0	0
Successful read that consumed the last byte	1	0	0	0
Read attempted at end of input	0	1	1	0
Malformed input (letters into an int)	0	0	1	0
<code>open()</code> on a missing file	0	0	1	0
Device/buffer failure	0	0	1	1

BEST PRACTICES

1. Use `setf(flag, mask)`, never bare `setf(flag)`, for flags in a group
2. Prefer manipulators to raw flag manipulation in ordinary code
3. Use `copyfmt` or an RAI guard to restore formatting in any function taking an `ostream&`
4. Test the operation, not `eof()`
5. Keep `eofbit` out of the exception mask
6. `clear()` before retrying anything on a failed stream
7. Distinguish `fail()` from `bad()` before deciding to recover
8. Use `xalloc` once, in a function-local `static`, so the index is allocated exactly once
9. Register a cleanup callback for anything you store in `pword`
10. `sync_with_stdio(false)` only when you have stopped using C `stdio`
11. Use `widen`/`narrow` in templates over character types
12. Reserve `unitbuf` for logs where losing buffered output would matter

RELATED HEADERS

```
#include <ios>           // ios_base, basic_ios, flags, io_errc, streamsize
#include <iosfwd>         // forward declarations only – cheap includes in headers
#include <iostream>       // pulls in <ios>, <istream>, <ostream> + the global streams
#include <streambuf>      // basic_streambuf – the buffer layer below basic_ios
#include <locale>         // facets that ios_base::imbue installs
#include <system_error>    // error_code, system_error (base of ios_base::failure)
```

EXTERNAL RESOURCES

- `std::ios_base`: https://en.cppreference.com/w/cpp/io/ios_base
- `std::basic_ios`: https://en.cppreference.com/w/cpp/io/basic_ios
- `<ios>`: <https://en.cppreference.com/w/cpp/header/ios>

Standard: C++11 (`io_errc`, `failure : system_error`) / C++23 (`noreplace`) **Last Updated:** September 2026